

91. Decode Ways - Explanation

Description

A string consisting of uppercase english characters can be encoded to a number using the following mapping:

'A' -> "1"
'B' -> "2"
...
'Z' -> "26"

To decode a message, digits must be grouped and then mapped back into letters using the reverse of the mapping above. There may be multiple ways to decode a message. For example, "1012" can be mapped into:

- "JAB" with the grouping (10 1 2)
- "JL" with the grouping (10 12)

The grouping (1 01 2) is invalid because 01 cannot be mapped into a letter since it contains a leading zero.

Given a string *s* containing only digits, return the number of ways to decode it. You can assume that the answer fits in a 32-bit integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation: "12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "01"

Output: 0

Explanation: "01" cannot be decoded because "01" cannot be mapped into a letter.

Constraints:

- $1 \leq s.length \leq 100$
- *s* consists of digits

You should aim for a solution as good or better than $O(n)$ time and $O(n)$ space, where *n* is the length of the given string.

The characters A through Z are mapped to the numbers from 1 to 26. A mapped number can have at most 2 digits. In the given string of digits, we can explore all possible decodings by combining one or two consecutive digits. Think of this in terms of a decision tree and explore all paths. Can you derive a recurrence relation for this?

Iterate over the string with index i . At each index, we have two choices: decode the current digit as a character with its mapped value, or combine the current digit with the next digit to form a two-digit value. The recurrence relation can be expressed as $\text{dfs}(i + 1) + \text{dfs}(i + 2)$ where dfs is the recursive function. Also, consider edge cases, as not every two-digit number or a number with a leading zero is valid. How would you implement this?

A brute-force recursive solution would result in $O(2^n)$ time complexity. Can you think of a better way? Perhaps you should consider the repeated work of calling the recursion multiple times with the same parameter values and find a way to avoid this. Also, can you think about the base condition of this recursive function?

The base condition is to return 1 if i goes out of bounds. If the current digit is '0', return 0, as no character maps to '0', making the string invalid. Use memoization to avoid repeated work by caching recursion results in an array or hash map and returning the stored value when the result is already calculated.

1. Recursion

Intuition

Each digit (or pair of digits) in the string can be mapped to a letter:

● "1" → "A", "2" → "B", ..., "26" → "Z"

At any index i , you only have two possible decoding choices:

1. Take one digit ($s[i]$) → valid if it's not '0'
2. Take two digits ($s[i:i+2]$) → valid if it forms a number between 10 and 26

So the problem naturally breaks into subproblems:

"How many ways can I decode the substring starting at index i ?"

This leads directly to a recursive structure.

Key base ideas:

- If you reach the end of the string → 1 valid decoding
- If a substring starts with '0' → 0 ways (invalid)
- Otherwise, sum the ways from:
 - decoding one digit
 - decoding two digits (if valid)

Algorithm

1. Define a recursive function $\text{dfs}(i)$ = number of ways to decode $s[i:]$.
2. Base cases:
 - If $i == \text{len}(s)$, return 1 (successfully decoded entire string).
 - If $s[i] == '0'$, return 0 (invalid decoding).
3. Recursively:
 - Always try decoding one digit → $\text{dfs}(i + 1)$
 - If two digits form a valid number (10–26), also try → $\text{dfs}(i + 2)$
4. Return the sum of valid choices.
5. Start recursion from index 0.

```
public class Solution {
    private int dfs(int i, String s) {
        if (i == s.length()) return 1;
        if (s.charAt(i) == '0') return 0;

        int res = dfs(i + 1, s);
        if (i < s.length() - 1) {
            if (s.charAt(i) == '1' ||
                (s.charAt(i) == '2' && s.charAt(i + 1) < '7')) {
                res += dfs(i + 2, s);
            }
        }
        return res;
    }

    public int numDecodings(String s) {
        return dfs(0, s);
    }
}
```

Time & Space Complexity

- Time complexity: $O(2n)$
 - Space complexity: $O(n)$
-

2. Dynamic Programming (Top-Down)

Intuition

This is the same decoding logic as the recursive approach, but with memoization to avoid recomputing the same subproblems.

Observation:

- While decoding, the same index i is reached multiple times.
- The number of ways to decode $s[i:]$ never changes, so we can store it once and reuse it.

So instead of recalculating:

“How many ways can I decode from index i ?”

we cache the answer the first time we compute it.

This converts the exponential recursion into linear time.

Algorithm

1. Use a dictionary dp where $dp[i]$ = number of ways to decode $s[i:]$.
2. Initialize base case:
 - $dp[\text{len}(s)] = 1$ (empty string has one valid decoding).
3. Define recursive function $\text{dfs}(i)$:
 - If i already in dp , return $dp[i]$.
 - If $s[i] == '0'$, return 0 (invalid).
4. Compute:
 - Take one digit $\rightarrow \text{dfs}(i + 1)$
 - Take two digits if valid (10–26) $\rightarrow \text{dfs}(i + 2)$
5. Store result in $dp[i]$ and return it.
6. Call $\text{dfs}(0)$.

```
public class Solution {  
    public int numDecodings(String s) {  
        Map<Integer, Integer> dp = new HashMap<>();  
        dp.put(s.length(), 1);  
        return dfs(s, 0, dp);  
    }  
    private int dfs(String s, int i, Map<Integer, Integer> dp) {  
        if (dp.containsKey(i)) {  
            return dp.get(i);  
        }  
        if (s.charAt(i) == '0') {  
            return 0;  
        }  
        int oneDigit = dfs(s, i + 1, dp);  
        int twoDigits = 0;  
        if (i + 2 <= s.length() && (s.charAt(i) < '3' || s.charAt(i) == '3' && s.charAt(i + 1) < '7')) {  
            twoDigits = dfs(s, i + 2, dp);  
        }  
        dp.put(i, oneDigit + twoDigits);  
        return dp.get(i);  
    }  
}
```

```

    }
    if (s.charAt(i) == '0') {
        return 0;
    }

    int res = dfs(s, i + 1, dp);
    if (i + 1 < s.length() && (s.charAt(i) == '1' ||
        s.charAt(i) == '2' && s.charAt(i + 1) < '7')) {
        res += dfs(s, i + 2, dp);
    }
    dp.put(i, res);
    return res;
}
}

```

Time & Space Complexity

- Time complexity: $O(n)$
 - Space complexity: $O(n)$
-

3. Dynamic Programming (Bottom-Up)

Intuition

This is the iterative version of the decoding logic.

Instead of asking:

“How many ways can I decode starting at index i ?”

recursively, we build the answer from the back.

Key idea:

- Let $dp[i]$ = number of ways to decode the substring $s[i:]$
- The answer we want is $dp[0]$
- Each position depends only on the next one or two positions → perfect for bottom-up DP

Algorithm

1. Create a DP table (or map) where:
 - $dp[i]$ = number of ways to decode $s[i:]$
2. Base case:
 - $dp[\text{len}(s)] = 1$ (empty string has one valid decoding)
3. Iterate i from right to left:

- If $s[i] == '0' \rightarrow dp[i] = 0$ (invalid start)
- Else:
 - Take one digit $\rightarrow dp[i] = dp[i + 1]$
 - Take two digits if valid (10–26) $\rightarrow$ add $dp[i + 2]$

4. Return $dp[0]$

```
public class Solution {
    public int numDecodings(String s) {
        int[] dp = new int[s.length() + 1];
        dp[s.length()] = 1;
        for (int i = s.length() - 1; i >= 0; i--) {
            if (s.charAt(i) == '0') {
                dp[i] = 0;
            } else {
                dp[i] = dp[i + 1];
                if (i + 1 < s.length() && (s.charAt(i) == '1' ||
                    s.charAt(i) == '2' && s.charAt(i + 1) < '7')) {
                    dp[i] += dp[i + 2];
                }
            }
        }
        return dp[0];
    }
}
```

Time & Space Complexity

- Time complexity: $O(n)$
 - Space complexity: $O(n)$
-

4. Dynamic Programming (Space Optimized)

Intuition

This is the space-optimized version of bottom-up DP.

From the bottom-up approach, we know:

- $dp[i]$ depends only on $dp[i+1]$ and $dp[i+2]$
- So we don't need an entire DP array

We just keep:

- $dp1 \rightarrow$ ways to decode from $i+1$
- $dp2 \rightarrow$ ways to decode from $i+2$

At each index i , we compute the current answer using these two values, then shift them forward.

Algorithm

1. Initialize:

- $dp1 = 1 \rightarrow$ corresponds to $dp[\text{len}(s)]$
- $dp2 = 0$

2. Iterate from right to left:

- If $s[i] == '0' \rightarrow$ current ways = 0
- Else:
 - Start with $dp1$ (take one digit)
 - If two digits form a valid number (10–26), add $dp2$

3. After computing current:

- Shift values:
 - $dp2 = dp1$
 - $dp1 = \text{current}$

4. Return $dp1$

```
public class Solution {
    public int numDecodings(String s) {
        int dp = 0, dp2 = 0;
        int dp1 = 1;
        for (int i = s.length() - 1; i >= 0; i--) {
            if (s.charAt(i) == '0') {
                dp = 0;
            } else {
                dp = dp1;
                if (i + 1 < s.length() && (s.charAt(i) == '1' ||
                    s.charAt(i) == '2' && s.charAt(i + 1) < '7')) {
                    dp += dp2;
                }
                dp2 = dp1;
                dp1 = dp;
            }
        }
        return dp1;
    }
}
```

Time & Space Complexity

- Time complexity: $O(n)$
- Space complexity: $O(1)$